

Python Interview Questions

Top 30 Most Asked -- With Answers and Code Examples

Categories

Core

OOP

Advanced

Concurrency

Data Structures

1 What is the difference between a list and a tuple?

Core

Lists are mutable (can be changed after creation) while tuples are immutable. Tuples are generally faster and can be used as dictionary keys. Lists allow appending, removing, or modifying elements.

```
# List -- mutable
lst = [1, 2, 3]
lst.append(4)      # [1, 2, 3, 4]
```

```
lst[0] = 99          # [99, 2, 3, 4]

# Tuple -- immutable
tpl = (1, 2, 3)
# tpl[0] = 99        # TypeError!
d = {'tpl': 'value'} # valid as dict key
```

2 How does Python manage memory? Explain reference counting and the GC.

Core

Python uses reference counting as its primary mechanism. Every object has a count of references; when it reaches zero the memory is freed. For cyclic references, the cyclic garbage collector periodically detects and collects cycles that reference counting alone cannot handle.

```
import sys, gc

x = [1, 2, 3]
print(sys.getrefcount(x)) # 2

# Cyclic reference -- GC handles this
a = []
b = [a]
a.append(b)
del a, b
gc.collect() # explicitly run cyclic GC
```

3 What are Python decorators? How do you write one?

Core

A decorator is a higher-order function that wraps another function to add behaviour without modifying the original. Applied with the `@` syntax, they are commonly used for logging, timing, access control, and caching.

```
import functools, time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - t0
        print(f"{func.__name__} took {elapsed:.4f}s")
        return result
    return wrapper

@timer
def slow(n):
    return sum(range(n))

slow(10_000_000) # "slow took 0.312s"
```

4 What is a generator and how does it differ from a list comprehension?

Core

A generator is a lazy iterator that yields values one at a time using the yield keyword, consuming $O(1)$ memory regardless of size. A list comprehension eagerly builds the entire list in memory. Use generators for large or infinite sequences.

```
# List comprehension -- eager, all in memory
sq_list = [x*x for x in range(10_000_000)] # ~80 MB

# Generator expression -- lazy, O(1) memory
sq_gen = (x*x for x in range(10_000_000)) # ~120 bytes

def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
print([next(fib) for _ in range(8)]) # [0,1,1,2,3,5,8,13]
```

5 Explain Python's GIL -- what it is and why it matters.

Core

The Global Interpreter Lock (GIL) is a mutex in CPython that allows only one thread to execute Python bytecode at a time. CPU-bound multithreaded programs don't get a true speedup. I/O-bound programs still benefit from threads because the GIL is released during I/O. Use multiprocessing for CPU parallelism.

```
from multiprocessing import Pool

def crunch(n): return sum(range(n))

with Pool(4) as p:
    results = p.map(crunch, [10**7]*4)

# I/O-bound: threading is fine
import threading
threads = [threading.Thread(target=fetch, args=(u,)) for u in urls]
[t.start() for t in threads]
[t.join() for t in threads]
```

6 What is the difference between == and is?

Core

`==` tests value equality (calls `__eq__`). `is` tests identity -- whether two names point to the exact same object in memory. Small integers (-5 to 256) and interned strings are cached, so "is" may unexpectedly return True. Never rely on `is` for value comparison.

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True (same values)
print(a is b) # False (different objects)
```

```
x = 256; y = 256
print(x is y)    # True   (cached)
x = 257; y = 257
print(x is y)    # False  (CPython-specific)

if value is None: ... # correct idiom
```

7 What are *args and **kwargs?

Core

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dict. They allow writing flexible functions that accept any number of arguments.

```
def demo(*args, **kwargs):
    print(args)    # tuple of positional args
    print(kwargs)  # dict of keyword args

demo(1, 2, 3, x=10, y=20)
# (1, 2, 3)
# {'x': 10, 'y': 20}

# Unpacking into a call
nums = [1, 2, 3]
opts = {"sep": "-"}
print(*nums, **opts) # 1-2-3
```

8 What is the difference between deepcopy and copy?

Core

copy.copy() creates a shallow copy -- a new object but nested objects are still shared references. copy.deepcopy() recursively copies all nested objects so the copy is fully independent.

```
import copy

orig = [[1, 2], [3, 4]]

shallow = copy.copy(orig)
shallow[0].append(99)
print(orig)    # [[1, 2, 99], [3, 4]] -- inner list shared!

orig = [[1, 2], [3, 4]]
deep = copy.deepcopy(orig)
deep[0].append(99)
print(orig)    # [[1, 2], [3, 4]] -- untouched
```

9 How does exception handling work in Python?

Core

Python uses try/except/else/finally. Code in else runs only if no exception was raised. finally always runs and is used for cleanup. Custom exceptions should inherit from Exception.

```

class InsufficientFunds(Exception):
    def __init__(self, amount, balance):
        super().__init__(f"Need {amount}, have {balance}")

def withdraw(bal, amt):
    if amt > bal:
        raise InsufficientFunds(amt, bal)
    return bal - amt

try:
    withdraw(100, 200)
except InsufficientFunds as e:
    print(e)
else:
    print("Success")    # only if no exception
finally:
    print("Done")       # always runs

```

10 What is the difference between mutable and immutable objects?

Core

Mutable objects (list, dict, set) can be changed in-place. Immutable objects (int, float, str, tuple, frozenset) cannot. A classic Python bug is using a mutable default argument -- it is shared across all calls.

```

# Mutable default argument -- classic bug!
def bad_append(val, lst=[]):    # lst shared across calls!
    lst.append(val)
    return lst

print(bad_append(1))    # [1]
print(bad_append(2))    # [1, 2] unexpected!

# Fix: use None as default
def good_append(val, lst=None):
    if lst is None: lst = []
    lst.append(val)
    return lst

```

11 Explain OOP in Python -- classes, inheritance, polymorphism.

OOP

Python supports encapsulation, inheritance, polymorphism, and abstraction. Use the abc module for abstract base classes. Python uses C3 linearization (MRO) to resolve multiple inheritance.

```

from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name): self.name = name
    @abstractmethod
    def speak(self): ...

class Dog(Animal):
    def speak(self): return f"{self.name} says Woof"

class Cat(Animal):

```

```
def speak(self): return f"{self.name} says Meow"

animals = [Dog("Rex"), Cat("Luna")]
for a in animals:
    print(a.speak())    # polymorphism
```

12 What are dunder (magic) methods? Give examples.

OOP

Dunder (double-underscore) methods let you define how objects behave with built-in operations -- arithmetic, comparison, printing, iteration, context managers, etc. They are called implicitly by Python syntax.

```
class Vector:
    def __init__(self, x, y): self.x, self.y = x, y
    def __repr__(self): return f"Vector({self.x}, {self.y})"
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)
    def __len__(self): return 2
    def __iter__(self):
        yield self.x; yield self.y

v1 = Vector(1, 2); v2 = Vector(3, 4)
print(v1 + v2)    # Vector(4, 6)
print(list(v1))   # [1, 2]
```

13 What is the @property decorator and when would you use it?

OOP

@property lets you define methods that are accessed like attributes. Use it to add validation or computed values while keeping a clean calling API.

```
class Temperature:
    def __init__(self, celsius=0): self._c = celsius

    @property
    def celsius(self): return self._c

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero")
        self._c = value

    @property
    def fahrenheit(self): return self._c * 9/5 + 32

t = Temperature(25)
print(t.fahrenheit)    # 77.0
t.celsius = -300        # ValueError
```

14 What is the difference between classmethod, staticmethod, and

OOP

instance method?

Instance methods receive self and access instance + class state. Class methods receive cls and are used as alternative constructors. Static methods receive neither -- utility functions logically grouped in the class.

```
class Date:
    def __init__(self, y, m, d): self.y, self.m, self.d = y, m, d

    def display(self):           # instance method
        return f"{self.y}-{self.m:02d}-{self.d:02d}"

    @classmethod
    def from_string(cls, s):    # alt constructor
        y, m, d = map(int, s.split("-"))
        return cls(y, m, d)

    @staticmethod
    def is_valid_year(y): return 1 <= y <= 9999

d = Date.from_string("2025-04-01")
print(d.display())           # 2025-04-01
```

15 What is a context manager and how do you implement one?

Advanced

A context manager wraps setup and teardown logic via the with statement. Implement using `__enter__`/`__exit__` on a class, or use `@contextlib.contextmanager` with a generator for simpler cases.

```
from contextlib import contextmanager
import time

class Timer:
    def __enter__(self):
        self.t = time.perf_counter(); return self
    def __exit__(self, *exc):
        self.elapsed = time.perf_counter() - self.t
        return False

@contextmanager
def timer():
    t = time.perf_counter()
    yield
    print(f"Elapsed: {time.perf_counter()-t:.4f}s")

with Timer() as t:
    _ = sum(range(10**7))
print(f"{t.elapsed:.4f}s")
```

16 How do you use `functools.lru_cache` and when should you?

Advanced

`@lru_cache` memoizes a function return values keyed by its arguments. Ideal for pure functions with repeated expensive calls (recursive DP, lookups). All arguments must be hashable. Use `maxsize=None` for unbounded cache.

```

from functools import lru_cache, cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

print(fib(50))          # instant
print(fib.cache_info()) # CacheInfo(hits=48, misses=51, ...)

# Python 3.9+: @cache is shorthand for @lru_cache(maxsize=None)
@cache
def fib2(n): return n if n < 2 else fib2(n-1) + fib2(n-2)

```

17 What are Python's async/await and how does the event loop work?

Advanced

async def defines a coroutine; await suspends it until a result is ready, yielding control to the event loop. The event loop manages I/O-bound concurrency without threads. Use asyncio.gather for concurrent coroutines.

```

import asyncio, aiohttp

async def fetch(session, url):
    async with session.get(url) as resp:
        return await resp.text()

async def main():
    urls = ["https://httpbin.org/get"] * 5
    async with aiohttp.ClientSession() as s:
        tasks = [fetch(s, u) for u in urls]
        results = await asyncio.gather(*tasks)
        print(f"Fetched {len(results)} responses")

asyncio.run(main())

```

18 Explain metaclasses in Python.

Advanced

A metaclass is the class of a class -- it controls how classes are created. type is the default metaclass. Custom metaclasses override __new__ or __init__ to add class-level behaviour like registering subclasses or enforcing interfaces.

```

class SingletonMeta(type):
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class Config(metaclass=SingletonMeta):
    def __init__(self): self.debug = False

a = Config(); b = Config()

```



```
print(a is b)    # True -- same object
```

19 What is the difference between `__new__` and `__init__`?

Advanced

`__new__` is called first and actually creates the instance (returns it). `__init__` then initialises it. Override `__new__` when subclassing immutable types (int, str, tuple) or implementing singletons.

```
class LimitedInt(int):
    def __new__(cls, value):
        if not (0 <= value <= 100):
            raise ValueError(f"{value} out of range [0,100]")
        return super().__new__(cls, value)

x = LimitedInt(50)
print(x + 10)    # 60 (behaves like int)
LimitedInt(200)  # ValueError
```

20 Explain Python's descriptor protocol.

Advanced

A descriptor is any object that defines `__get__`, `__set__`, or `__delete__`. Placed as class attributes, they intercept attribute access. Python property, staticmethod, classmethod, and function objects are all descriptors.

```
class Validated:
    def __set_name__(self, owner, name): self.name = name
    def __get__(self, obj, objtype=None):
        if obj is None: return self
        return obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        if not isinstance(value, (int, float)) or value < 0:
            raise ValueError(f"{self.name} must be non-negative")
        obj.__dict__[self.name] = value

class Product:
    price = Validated()
    quantity = Validated()

p = Product()
p.price = 9.99    # ok
p.quantity = -1   # ValueError
```

21 What is the time complexity of common list operations?

Data Structures

Lists are backed by a dynamic array. Key costs: indexing $O(1)$, append amortised $O(1)$, insert/delete at arbitrary index $O(n)$, membership test $O(n)$. For $O(1)$ membership use a set; for sorted insertion use bisect.

```
import bisect, timeit
```

```

lst = list(range(100_000))
st = set(range(100_000))

t_list = timeit.timeit(lambda: 99_999 in lst, number=1000)
t_set = timeit.timeit(lambda: 99_999 in st, number=1000)
print(f"list: {t_list:.4f}s set: {t_set:.6f}s")
# list ~5s vs set ~0.00006s

a = list(range(0, 1000, 2))
bisect.insort(a, 501) # O(log n) search + O(n) shift

```

22 How does Python's dict work internally?

Data Structures

A dict is a hash table. Python hashes the key, computes a slot index, and handles collisions with open addressing. Since Python 3.7, dicts maintain insertion order as a language guarantee. Load factor triggers $O(n)$ resizing, amortised to $O(1)$.

```

class Point:
    def __init__(self, x, y): self.x, self.y = x, y
    def __hash__(self): return hash((self.x, self.y))
    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

d = {Point(1,2): "origin"}
print(d[Point(1,2)]) # "origin"

# defaultdict avoids KeyError
from collections import defaultdict
freq = defaultdict(int)
for ch in "hello": freq[ch] += 1

```

23 Explain list, dict, and set comprehensions.

Data Structures

Comprehensions are concise, readable ways to build collections -- generally faster than equivalent for loops because the iteration happens at the C level. They support filtering with if and nesting.

```

words = ["hello", "world", "python", "interview"]

uppers = [w.upper() for w in words if len(w) > 5] # list
lengths = {w: len(w) for w in words} # dict
initials = {w[0] for w in words} # set

# Nested (flatten matrix)
matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat = [x for row in matrix for x in row]

# Generator -- no list built
total = sum(x*x for x in range(1000))

```

24 What is the collections module and when do you use it?

Data Structures

The collections module provides specialised container types: Counter (frequency counting), defaultdict (default values), deque (O(1) append/pop at both ends), and namedtuple for structured records.

```
from collections import Counter, deque, namedtuple

text = "the quick brown fox jumps over the lazy fox"
freq = Counter(text.split())
print(freq.most_common(2))  # [('the', 2), ('fox', 2)]

# deque -- O(1) popleft
q = deque([1, 2, 3])
q.appendleft(0); q.popleft()  # 0

Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4); print(p.x, p.y)  # 3 4
```

25 What is the difference between isinstance and type()?

Core

isinstance(obj, cls) checks if an object is an instance of a class or any of its subclasses -- the idiomatic way to do type checking. Avoid type(obj) == cls as it ignores inheritance.

```
class Animal: pass
class Dog(Animal): pass

d = Dog()
print(type(d) == Animal)      # False
print(type(d) == Dog)        # True
print(isinstance(d, Animal))  # True (inheritance!)
print(isinstance(d, Dog))     # True

print(isinstance(42, (int, float)))  # True
```

26 What are threads vs processes vs asyncio in Python?

Concurrency

Choose based on bottleneck: asyncio for many concurrent I/O operations (single thread); threading for I/O-bound work with blocking libraries; multiprocessing for CPU-bound parallelism (bypasses the GIL with separate processes).

```
# Threading -- I/O-bound
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=10) as ex:
    results = list(ex.map(fetch, urls))

# Multiprocessing -- CPU-bound
from concurrent.futures import ProcessPoolExecutor
with ProcessPoolExecutor() as ex:
    results = list(ex.map(crunch, chunks))

# asyncio -- high-concurrency I/O
import asyncio
async def main():
```

```
await asyncio.gather(*[fetch_async(u) for u in urls])
```

27 What is a lambda function and when should you use one?

Core

A lambda is an anonymous single-expression function. Use it for short throwaway functions passed as arguments (key= in sort, map, filter). For anything more complex, use a named def for readability.

```
people = [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}]
people.sort(key=lambda p: p["age"])

nums = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x*x, nums))
evens = list(filter(lambda x: x%2==0, nums))

# Prefer comprehensions -- more readable
squared = [x*x for x in nums]
evens = [x for x in nums if x % 2 == 0]
```

28 How do you profile Python code?

Advanced

Python provides cProfile (statistical CPU profiler, built-in), timeit (microbenchmarks), line_profiler (line-level), and memory_profiler (memory usage). Always profile before optimising -- intuition is usually wrong.

```
import cProfile, pstats, io

def slow_func():
    return sum(i*i for i in range(10**6))

pr = cProfile.Profile()
pr.enable()
slow_func()
pr.disable()

s = io.StringIO()
pstats.Stats(pr, stream=s).sort_stats("cumulative").print_stats(10)
print(s.getvalue())

import timeit
t = timeit.timeit("sum(range(1000))", number=10_000)
```

29 What are Python type hints and how do you use them?

Advanced

Type hints (PEP 484) let you annotate variables and functions for static analysis with mypy or pyright. Not enforced at runtime but improve IDE support and catch bugs early. Python 3.10+ adds X | Y union syntax.

```
from typing import Optional
from collections.abc import Sequence
```

```
def greet(name: str, times: int = 1) -> str:
    return (f"Hello, {name}! " * times).strip()

def first(items: Sequence[int]) -> Optional[int]:
    return items[0] if items else None

# Python 3.10+ union syntax
def parse(value: str | int | None) -> float:
    if value is None: return 0.0
    return float(value)

from dataclasses import dataclass
@dataclass
class Point:
    x: float; y: float; label: str = ""
```

30 What is the difference between `__str__` and `__repr__`?

Advanced

`__repr__` is for developers -- it should return an unambiguous string that ideally can recreate the object. `__str__` is for end users -- a readable string. If `__str__` is not defined, Python falls back to `__repr__`.

```
class Fraction:
    def __init__(self, num, den):
        self.num, self.den = num, den

    def __repr__(self):
        return f"Fraction({self.num}, {self.den})"

    def __str__(self):
        return f"{self.num}/{self.den}"

f = Fraction(3, 4)
print(repr(f)) # Fraction(3, 4)
print(str(f))  # 3/4
print(f)       # 3/4 (uses __str__)
```